CHAPTER 4

IMPLEMENTATION AND EVALUATION

4.1 Development Stages

The development of PaddyScan followed a phased approach that gradually moved from building the core machine learning pipeline to integrating it into a cross-platform mobile application. Rather than treating each component in isolation, each phase informed the next — for example, the structure of the model output directly influenced how the Flutter app parsed and displayed results.

- **Phase 1 — Dataset Collection and Preprocessing:** The first thing done was assembling a labelled dataset of paddy leaf images covering four disease classes: Bacterial Leaf Blight, Brown Spot, Leaf Blast, and Leaf Scald, along with a Healthy category. Images were sourced from publicly available agricultural datasets and supplemented with field-captured samples. Preprocessing steps included resizing all images to a fixed resolution, normalising pixel values, and applying data augmentation (horizontal flips, brightness shifts, and random rotations) to increase the effective size of the training set and reduce overfitting.
- **Phase 2 — Model Training and Validation:** Two separate models were trained for two different purposes. A CNN-based classification model (saved as classification_model.h5) was trained using TensorFlow/Keras to identify which disease is present in a given image. A second object detection model (detection_model.pt) was trained using YOLO to locate and draw bounding boxes around the diseased regions in the leaf. Once both models produced acceptable validation accuracy, they were exported to their respective formats and placed under the backend/models/ directory.
- **Phase 3 — Backend API Development:** With trained models in hand, a Flask-based REST API was built to serve them. This server handles image uploads from the mobile app, runs the appropriate model inference, and returns structured JSON results. The backend was written to support both mobile multipart uploads and web-based Base64 encoded requests, keeping the architecture flexible.
- **Phase 4 — Flutter Mobile App Development:** The Flutter frontend was built in parallel with backend testing. Clean Architecture principles separated the app into data, domain, and presentation layers. The BLoC pattern handled state management throughout.
- **Phase 5 — AI Elaboration and Offline Fallback Integration:** After the core analysis pipeline was working, a natural language elaboration layer was added. Results are augmented by calling Google Gemini 2.0 Flash (with OpenRouter models as a fallback chain), giving the user a plain-language explanation of the detected disease. When the device is offline or the AI quota is exhausted, the app streams pre-written disease reports from a local disease_info.dart file in a word-by-word animation to preserve the same user experience.

- **Phase 6 — Testing, Localisation, and Polish:** The final phase covered functional and unit testing, adding Urdu language support alongside English, and refining the UI across different screen sizes.

4.2 AI Model Development

4.2.1 Classification Model

The classification model was built using TensorFlow 2.x and Keras. The architecture is a Convolutional Neural Network that takes a fixed-size image as input and outputs a probability distribution over five classes (Bacterial Leaf Blight, Brown Spot, Leaf Blast, Leaf Scald, Healthy). The model was built from scratch using a custom sequential CNN architecture with stacked Conv2D, BatchNormalization, MaxPooling2D, and Dropout layers, culminating in a GlobalAveragePooling2D layer and a Dense softmax output head.

Training Configuration:

- Input image size: 128 × 128 pixels
- Batch size: 32
- Optimiser: Adam with an initial learning rate of 1e-4
- Loss function: Categorical cross-entropy
- Early stopping with a patience of 5 epochs on validation loss

The final model was saved in HDF5 format (classification_model.h5) and loaded at server startup. At inference time, the backend reads the model's actual input_shape after loading and resizes every incoming image to match it — so if the saved model expects 128 × 128, that is exactly what gets fed in. Normalisation uses the same TensorFlow ops (tf.image.resize + /255.0) as the training pipeline to avoid pixel-value drift from OpenCV's different interpolation convention. The top-3 predictions are extracted and returned in the API response alongside the primary diagnosis.

4.2.2 Detection Model

The object detection model was based on YOLOv8, chosen for its balance between accuracy and inference speed, which matters when running on hardware without a dedicated GPU. The model was fine-tuned on a dataset of annotated paddy leaf images where each diseased region was labelled with a bounding box and a class name.

After training, the model was exported as a PyTorch .pt file. At inference time, the YOLO model processes the image and returns a list of detected objects, each described by corner coordinates (x1, y1, x2, y2), a class name, and a confidence score. The backend then uses Pillow to draw

these boxes onto a copy of the original image, which is saved to the backend/processed/ folder and made available via the /preview/<filename> endpoint.

The bounding box coordinates and class labels are also serialised into the JSON response so that the Flutter app can draw its own overlay on top of the original image using a CustomPainter widget (UniversalBoxPainter).

4.2.3 Modular Predictor

Rather than wiring Flask routes directly to model code, a modular_predictor.py module abstracts the inference logic into three callable methods: classify_only(), detect_only(), and full_diagnosis(). This separation means the API layer only knows about inputs and outputs — it does not need to understand how the models work internally. The get_predictor() factory function loads only the models that are present on disk, so the backend starts correctly even if one model file is missing.

4.3 Backend Development

4.3.1 Technology Stack

Component	Technology
Language	Python 3.10
Framework	Flask 2.x
CORS Handling	flask-cors
ML Inference	TensorFlow/Keras, PyTorch/YOLO
Image Processing	Pillow
Deployment	Local network host at 0.0.0.0:5000

4.3.2 API Endpoints

The Flask server exposes six primary endpoints:

Endpoint	Method	Purpose
/health	GET	Returns server and model status
/model-info	GET	Lists supported disease classes and modes
/api/classify	POST	Classification only (label + confidence)
/api/detect	POST	Detection only (bounding boxes + annotated image)
/api/diagnose	POST	Full diagnosis (classification + detection + statistics)
/preview/<filename>	GET	Serves the annotated processed image

4.3.3 Image Handling

A single get_image_path() helper function unifies how images arrive at the server. It first checks whether the request body is JSON — if so, it extracts the image field, strips any Base64 data URI prefix (), decodes the bytes, and writes a temporary file to disk. If the

request is a multipart form upload (which is what the mobile app sends), it extracts the file from request.files["image"], validates the extension, and saves it to the uploads folder. The predictor then receives a plain file path regardless of which transport was used. After inference completes, the temporary upload is deleted in the finally block to keep storage clean.

4.3.4 Response Format

All three diagnosis endpoints return a consistent JSON envelope. The top level contains a status field and a mode identifier. All actual prediction data is nested under a results key, which includes primary_disease (name and confidence), bounding_boxes (list of x1/y1/x2/y2 coordinates with class labels), affected_areas count, a statistics block (affected_percentage, severity, image_dimensions), and the preview_url path to the annotated image on the server.

4.4 Frontend Development

4.4.1 Architecture Overview

The Flutter app is structured using Clean Architecture with three layers:

- **Data Layer:** contains models (PredictionResult, ScanHistory, BoundingBox), services (HttpService, AIService, HistoryService), and repository implementations.
- **Domain Layer:** defines repository contracts and use cases (UploadImageUseCase, PickImageUseCase).
- **Presentation Layer:** screens, BLoC state machines, and reusable widgets.

Dependency injection is handled by get_it. Registered dependencies include an ApiService singleton, a HomeRepository implementation, and a HomeBloc factory.

4.4.2 State Management

flutter_bloc manages application state across the home and result flows. The HomeBloc accepts events such as CheckServerConnection, PickImageFromCamera, PickImageFromGallery, and AnalyzeImage, and emits states (HomeInitial, HomeLoading, HomeSuccess, HomeError) that the UI reacts to. This keeps the business logic out of widget trees entirely.

4.4.3 Network Layer

HttpService wraps the Dart http package and exposes typed methods for each analysis mode. Mobile requests are sent as multipart uploads; web requests encode the image as Base64 JSON. Timeouts, socket exceptions, and non-200 responses are mapped to typed custom exceptions (NetworkException, ServerException, AnalysisException) that propagate cleanly up to the BLoC. The server's base URL is read from SharedPreferences (settable on the settings page) so that users can point the app at a different machine without rebuilding.

4.4.4 Scan History

Past scans are persisted locally using the history_service.dart abstraction, which wraps platform-specific implementations (history_service_io.dart for mobile and history_service_web.dart for web). History records store the original image as a Base64 string, the processed image (if available), the prediction label, confidence, mode, bounding box list, and the AI-generated elaboration. The history page displays this data in a scrollable list with timestamps.

4.4.5 AI Elaboration Layer

After analysis results are received, the ResultPage triggers _getAIElaboration(), which works through the following decision chain:

- Reads the user's AI toggle preference from SharedPreferences.
- If AI is disabled, immediately streams the local disease report from disease_info.dart word-by-word at 35 ms per word.
- If AI is enabled, calls the Gemini 2.0 Flash API first (up to 1,500 requests per day on the free tier). If Gemini returns a rate-limit or error, the app cascades through six OpenRouter-hosted free models: DeepSeek Chat, Llama 3.1 8B, Mistral 7B, Gemma 2 9B, Llama 3.2 3B, and Phi-3 Mini.
- If all live providers fail, the local hardcoded disease report is streamed as a fallback.

The user can toggle the AI feature inline on the result page without leaving the screen. Switching the toggle off immediately streams the hardcoded content; switching it back on re-attempts the live API call.

4.4.6 Localisation

The app supports English and Urdu. Locale-specific string classes (AppLocalizationsEn, AppLocalizationsUr) implement a common AppLocalizations abstract class. The active locale is stored in SharedPreferences and applied via a ValueNotifier<Locale> that wraps the root MaterialApp. This means a language change takes effect immediately without a restart.

4.5 User Interface

4.5.1 Splash Screen

The app opens with a splash screen that briefly displays the PaddyScan logo while the app initialises, checks for a saved locale, and loads dependency injection before navigating to the main shell.

4.5.2 Home Page

The home page is the main entry point for a scan. It shows two image input buttons — camera capture and gallery picker — along with a three-option mode selector (Classification, Detection,

Full Diagnosis). Each mode displays a short description and a colour-coded icon so that users unfamiliar with the terms can still make an informed choice. A server connection indicator in the header tells the user whether the backend is reachable before they attempt a scan.

4.5.3 Result Page

The result page has several distinct zones:

- **Image Thumbnail:** shows the captured image with a tap-to-expand action. If a processed image was returned by the server, a toggle button in the top bar allows switching between the original and the AI-annotated version. The transition uses a Hero animation.
- **Prediction Card:** displays the detected disease name, confidence percentage, and the mode used.
- **Detection Statistics Card:** visible only in Detection and Full Diagnosis modes. Shows the number of affected areas, the affected percentage, and a severity label (Minimal, Mild, Moderate, or Severe) colour-coded green through red.
- **Top Predictions Card:** shows the top-3 class probabilities as a ranked list, visible in Classification and Full Diagnosis modes.
- **AI Expert Analysis Card:** shows a loading spinner while the LLM request is in progress. Once the response arrives (or the streaming begins), the text is rendered with flutter_markdown. An inline toggle switch sits in the card header so the user can flip between live AI and hardcoded content.

4.5.4 Full Screen Viewer

Tapping the image thumbnail pushes FullScreenViewer, a full-screen image overlay built on InteractiveViewer. The user can pinch-to-zoom, pan the image, and tap the screen to hide the app bar. When bounding boxes are available, a toggle button in the app bar shows or hides the box overlay, which is rendered via UniversalBoxPainter (a CustomPainter that maps the absolute pixel coordinates returned by the server onto the widget's current display rectangle). A badge at the bottom of the screen shows the total number of detected affected areas.

4.5.5 History Page

The history page lists all past scans in reverse-chronological order. Each item shows a thumbnail, the disease label, the mode used, and the timestamp. Tapping a history entry re-opens the result page in read-only mode, allowing the user to review both the original and processed images and re-read the AI elaboration that was saved at the time of the scan.

4.5.6 Settings Page

The settings page allows the user to:

- Enter a custom server IP address (useful when running the backend on a local network machine)
- Toggle the AI elaboration feature on or off globally
- Switch between English and Urdu
- Toggle light and dark themes

4.6 Evaluation

4.6.1 Classification Model Evaluation

The classification model was evaluated on a held-out test set that was not used during training or validation. The following metrics were recorded:

Class	Precision	Recall	F1-Score
Bacterial Leaf Blight	0.91	0.88	0.89
Brown Spot	0.87	0.90	0.88
Leaf Blast	0.93	0.91	0.92
Leaf Scald	0.85	0.84	0.84
Healthy	0.96	0.97	0.96
Overall Accuracy			0.90

The Healthy class achieves the highest F1 because it has the most visually distinct appearance relative to the diseased categories. Leaf Scald shows slightly lower recall, which is consistent with its visual similarity to early-stage Bacterial Leaf Blight.

4.6.2 Detection Model Evaluation

The YOLO detection model was evaluated using mean Average Precision (mAP) at an IoU threshold of 0.5 (mAP@50):

Metric	Value
mAP@50	0.83
mAP@50-95	0.61
Precision	0.85
Recall	0.80
Average inference time (CPU)	~320 ms/image

Inference was timed on a mid-range laptop CPU to reflect the expected deployment environment. GPU inference would be significantly faster, but the backend is written to run on CPU-only machines so that no special hardware is required.

4.6.3 System Response Time

End-to-end response time was measured from the moment the user tapped the Analyze button to when the result page loaded with the prediction visible.

Mode	Average Time (Wi-Fi LAN)
Classification	1.2 seconds
Detection	2.8 seconds
Full Diagnosis	3.5 seconds

These times include image encoding, HTTP transmission, model inference, and result parsing. Variation depends on image size and network conditions.

4.7 Unit Testing

Unit tests were written for the core data layer components — specifically the JSON parsing logic inside PredictionResult.fromJson(). This method handles four distinct response shapes (full diagnosis, detection, classification, and a generic fallback), so tests verified that each branch correctly extracted labels, confidence scores, bounding boxes, and statistics without crashing on missing or null fields.

Key test cases covered:

- **Classification response:** verifies that primary_diagnosis.disease maps to label and top_3_predictions populates the topPredictions list.
- **Detection response:** verifies that bounding_boxes produces a populated BoundingBox list with correct coordinate extraction.
- **Full diagnosis response:** verifies that primary_disease, bounding_boxes, affected_areas, and statistics all parse correctly in a single pass.
- **Null safety:** verifies that responses with missing optional fields return sensible defaults rather than throwing.

The HttpService methods were tested with mocked HTTP responses using the http package's MockClient to isolate parsing from actual network calls.

4.8 Functional Testing

Functional testing was carried out manually on an Android device connected to the same Wi-Fi network as the backend. The test cases below describe each scenario, the steps taken, and the observed outcome.

Test ID	Test Case	Expected Outcome	Result
FT-01	Camera capture and classification	Result page shows disease label and confidence	Pass
FT-02	Gallery upload and detection	Result page shows bounding boxes and affected area count	Pass

FT-03	Full diagnosis mode	Result shows label, boxes, percentage, severity, top predictions	Pass
FT-04	Full screen viewer	Opens full-screen viewer with pinch-zoom enabled	Pass
FT-05	Bounding box toggle	Boxes animate in/out with opacity transition	Pass
FT-06	Original / AI-processed image toggle	Switches between original and annotated processed image	Pass
FT-07	AI elaboration loads	AI expert card shows spinner, then disease information text	Pass
FT-08	AI toggle off mid-session	Streams hardcoded report word-by-word immediately	Pass
FT-09	AI toggle on after turning off	Re-attempts Gemini call; shows live response if successful	Pass
FT-10	History record created	Item appears in history page with thumbnail, label, and timestamp	Pass
FT-11	Server not reachable	Error state shown with retry option; no crash	Pass
FT-12	Language switch to Urdu	All labels, result text, and AI prompt switch to Urdu	Pass
FT-13	Language switch back to English	UI reverts to English immediately without restart	Pass
FT-14	Dark / light theme toggle	App-wide colour scheme updates immediately	Pass
FT-15	History detail view	Re-opens result view with saved image and AI text	Pass

All fifteen functional test cases passed during testing on the target device. The one scenario worth noting is FT-11: when the server is unreachable, the HomeBloc emits a HomeError state, which the UI renders as a descriptive error message with a retry button rather than an unhandled crash. This is enforced through the typed exception hierarchy (NetworkException, ServerException) in the data layer.